📄 HLD — High-Level Design

RAG-Based Customer Support Assistant

1. System Overview

1.1 Problem Definition

Customer support systems often handle a large number of repetitive queries related to product returns, refunds, shipping, order cancellation, and payment issues. Traditional support systems require human agents to manually respond to these queries, which increases response time and operational costs.

Customers expect quick and accurate responses, but human support teams may experience delays due to high workload. Additionally, manually searching through documents such as policies and FAQs can be time-consuming.

To address these challenges, an automated system is required that can retrieve relevant information from company documents and generate accurate responses to customer queries.

1.2 Proposed Solution

The proposed system is a Retrieval-Augmented Generation (RAG)-based Customer Support Assistant designed to automate responses to customer queries using information stored in a PDF knowledge base.

The system processes customer queries by retrieving relevant content from stored documents and generating context-aware responses using a language model.

The solution also includes:

Document ingestion pipeline

Vector-based search using embeddings

Workflow orchestration using LangGraph

Conditional routing based on query understanding

Human-in-the-Loop (HITL) escalation for complex cases

This system improves response speed, accuracy, and customer satisfaction.

1.3 Scope of the System

The system focuses on handling customer queries related to:

Product returns

Refund requests

Shipping details

Payment issues

Order cancellation

Warranty information

The system supports:

Uploading PDF knowledge base

Processing user queries

Retrieving relevant information

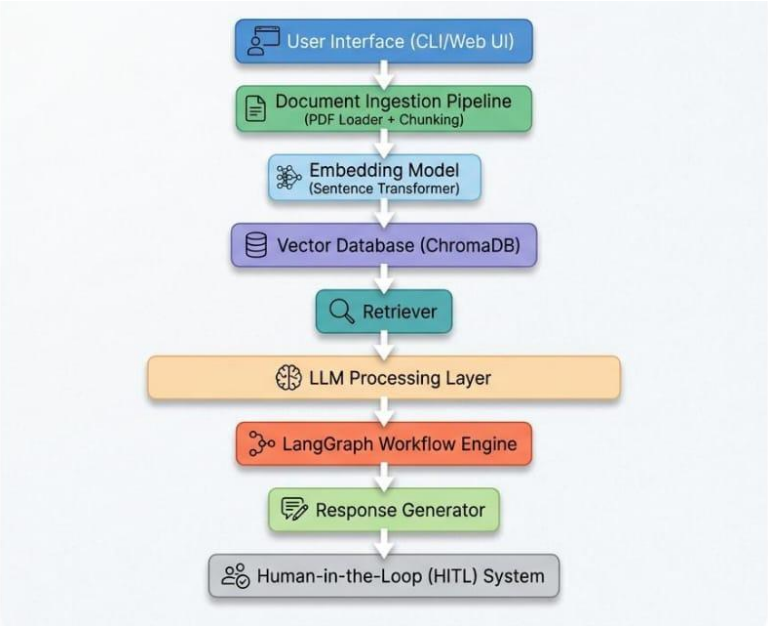
Generating accurate responses

Escalating complex queries to human agents

The system operates as a customer support assistant that reduces manual effort and improves support efficiency.

---

2. System Architecture



2.1 Architecture Overview

The architecture of the RAG-Based Customer Support Assistant consists of multiple interconnected components designed to process documents, retrieve relevant information, and generate accurate responses to customer queries.

The system begins with the **User Interface,** where users submit their queries through a command-line interface or web interface. The system receives the query and sends it to the document processing pipeline.

The **Document Ingestion Pipeline** loads the PDF knowledge base and splits the document into smaller text chunks. This step ensures that large documents can be processed efficiently.

These text chunks are passed to the **Embedding Model,** which converts each chunk into numerical vector representations. These vectors capture the semantic meaning of the text.

The generated embeddings are stored in the **Vector Database (ChromaDB).** This database enables fast similarity-based search during query processing.

When a user submits a query, the **Retriever** searches the vector database and retrieves the most relevant chunks related to the query.

The retrieved content is passed to the **LLM Processing Layer,** where the language model generates a meaningful response using the retrieved information as context.

The **LangGraph Workflow Engine** manages the overall execution flow of the system. It controls decision-making logic and determines how queries should be processed.

If the system detects low confidence or complex queries, the request is forwarded to the **Human-in-the-Loop (HITL)** module. This module allows human agents to review and respond to difficult queries.

This architecture ensures fast retrieval, accurate response generation, and reliable escalation support.

2.2 Data Flow

The data flow in the RAG-Based Customer Support Assistant begins with document ingestion and ends with response generation.

First, the system loads the customer support PDF document using the document loader. The document is split into smaller text chunks using a chunking strategy. These chunks allow efficient processing of large documents.

Next, each chunk is converted into embeddings using an embedding model. These embeddings represent the semantic meaning of the text.

The embeddings are stored in the vector database, which allows similarity-based retrieval.

When a user submits a query, the query is also converted into an embedding vector. The retriever searches the vector database to find the most relevant text chunks.

The retrieved chunks are passed to the language model, which generates a response based on the provided context.

If the response confidence is low or the query is complex, the system forwards the request to the Human-in-the-Loop module for manual review.

Finally, the generated response is returned to the user.

---

3. Component Description

This section describes the major components used in the RAG-Based Customer Support Assistant system. Each component plays a specific role in processing documents, retrieving information, generating responses, and managing workflow operations.

3.1 Document Loader

The Document Loader is responsible for loading the knowledge base document into the system. In this project, the knowledge base is stored in PDF format containing customer support policies such as returns, refunds, shipping, and payment information.

The document loader reads the PDF file and extracts text content from each page. This extracted text becomes the input for further processing stages such as chunking and embedding generation.

This component ensures that the knowledge base content is properly imported into the system for processing.

3.2 Chunking Strategy

The Chunking Strategy divides large text documents into smaller segments called chunks. Chunking is necessary because large documents cannot be processed efficiently as a whole.

Each chunk typically contains a fixed number of characters or words. Smaller chunks help improve retrieval accuracy and reduce processing time.

Chunking also ensures that meaningful portions of text are preserved, allowing the system to retrieve relevant context when answering queries.

3.3 Embedding Model

The Embedding Model converts text chunks into numerical vector representations known as embeddings. These embeddings capture the semantic meaning of the text.

In this project, a sentence transformer model is used to generate embeddings. These embeddings allow the system to compare text based on meaning rather than exact keyword matching.

This component plays a critical role in enabling similarity-based search within the vector database.

3.4 Vector Database (ChromaDB)

The Vector Database stores the embeddings generated from text chunks. In this project, ChromaDB is used as the vector storage system.

Each stored embedding corresponds to a specific chunk of text. When a query is received, the system searches this database to find embeddings that are most similar to the query embedding.

ChromaDB supports fast retrieval and efficient storage of large numbers of embeddings.

3.5 Retriever

The Retriever is responsible for searching the vector database and retrieving relevant text chunks based on similarity.

When a user submits a query, the retriever converts the query into an embedding vector. It then compares this vector with stored embeddings to identify the most relevant chunks.

The retrieved chunks are passed to the language model to generate a context-based response.

3.6 Language Model (LLM)

The Language Model processes retrieved content and generates meaningful responses.

In this project, a language model is used to understand the user query and produce responses using the retrieved context.

The model combines the user query with retrieved text chunks and generates a natural language response.

This component ensures that responses are accurate, relevant, and understandable.

3.7 LangGraph Workflow Engine

The LangGraph Workflow Engine manages the overall system flow. It controls how different components interact with each other.

LangGraph defines nodes and transitions between steps such as query processing, response generation, and escalation.

This workflow engine ensures structured execution and improves system reliability.

3.8 Routing Layer

The Routing Layer determines how queries are processed based on their complexity.

If the system can generate a reliable response, the query is processed normally. However, if the system detects uncertainty or missing information, the query is routed to the Human-in-the-Loop system.

This decision-making mechanism improves response accuracy and ensures that critical queries are handled appropriately.

3.9 Human-in-the-Loop (HITL) Module

The Human-in-the-Loop (HITL) Module handles complex or uncertain queries that require manual intervention.

If the system cannot confidently generate a response, the query is escalated to a human support agent.

The human agent reviews the query and provides a final response. This ensures that difficult or sensitive cases are handled correctly.

This component enhances system reliability and customer satisfaction.

4. Technology Choices

This section describes the technologies selected for implementing the RAG-Based Customer Support Assistant system. Each technology is chosen based on performance, flexibility, scalability, and compatibility with modern AI workflows.

4.1 Programming Language — Python

Python is selected as the primary programming language for this project due to its simplicity, readability, and wide support for machine learning and natural language processing libraries.

Python provides extensive libraries such as LangChain, Hugging Face, and PyTorch, which simplify the development of AI-based systems. It also allows rapid prototyping and integration with multiple AI tools.

The availability of strong community support and documentation makes Python suitable for building scalable NLP systems.

4.2 Document Processing — PyPDF

PyPDF is used to load and extract text content from PDF documents. This library enables efficient reading of PDF files and allows extraction of text from multiple pages.

Since the system relies on a PDF knowledge base, PyPDF ensures that document content can be easily processed and converted into chunks.

4.3 Chunking Strategy — Recursive Text Splitting

Recursive text splitting is used to divide large documents into smaller chunks. This method preserves sentence structure and maintains meaningful context within each chunk.

Chunking improves retrieval accuracy and reduces processing complexity by enabling smaller units of text to be stored and searched efficiently.

4.4 Embedding Model — Sentence Transformers

Sentence Transformers are used to generate embeddings from text chunks. These models convert text into dense vector representations that capture semantic meaning.

Embedding models allow the system to retrieve contextually similar information rather than relying only on keyword matching.

Sentence Transformers are widely used in semantic search systems due to their high accuracy and efficiency.

4.5 Vector Database — ChromaDB

ChromaDB is selected as the vector database for storing embeddings.

It provides fast similarity search, persistent storage, and easy integration with Python-based frameworks. ChromaDB supports large datasets and allows efficient retrieval of relevant chunks.

Its lightweight nature makes it suitable for small to medium-scale AI applications.

4.6 Retrieval Framework — LangChain

LangChain is used to manage document retrieval and query processing operations.

It provides built-in tools for loading documents, splitting text, generating embeddings, and performing similarity search. LangChain simplifies the development of Retrieval-Augmented Generation systems.

This framework reduces coding complexity and improves workflow integration.

4.7 Workflow Engine — LangGraph

LangGraph is used to define the workflow structure of the system.

It enables the creation of graph-based workflows where nodes represent processing steps and edges represent transitions between steps.

LangGraph helps implement conditional routing logic and supports complex workflows such as Human-in-the-Loop escalation.

This makes the system more structured and easier to manage.

4.8 Language Model (LLM)

A language model is used to generate responses based on retrieved information.

The LLM processes user queries and contextual information to generate meaningful and accurate responses. The model enables natural language understanding and improves user interaction quality.

Language models enhance the system's ability to respond to complex customer queries.

4.9 User Interface — Command Line Interface (CLI)

A Command Line Interface (CLI) is used as the user interface for interacting with the system.

The CLI allows users to input queries and receive responses in a simple text-based format. This approach simplifies development and testing without requiring web deployment.

Future versions of the system may include a web-based interface.

---

5. Scalability Considerations

Scalability is an important factor in designing modern AI-based systems. The RAG-Based Customer Support Assistant must be capable of handling large documents, increasing numbers of user queries, and maintaining fast response times.

This section describes how the system can scale efficiently as data and user demand increase.

5.1 Handling Large Documents

As the size of the knowledge base increases, the system must efficiently process large volumes of text.

To handle large documents, the system uses a chunking strategy that divides documents into smaller text segments. This ensures that large files do not overload the system memory and allows efficient processing of content.

Additionally, embeddings are stored in a vector database, which supports efficient indexing and retrieval even when the number of stored chunks increases significantly.

This approach ensures that the system can scale to support multiple documents without performance degradation.

5.2 Increasing Query Load

As the number of users increases, the system must handle multiple queries simultaneously.

To support higher query loads, the system can implement parallel processing, allowing multiple requests to be processed at the same time. The vector database supports fast similarity search, enabling quick retrieval even under heavy usage.

Load-balancing techniques can also be applied to distribute queries across multiple processing units.

These strategies ensure consistent system performance as user demand grows.

5.3 Reducing Latency

Latency refers to the time required to generate a response after receiving a user query.

To reduce latency, the system uses optimized embedding models and efficient vector search techniques. Storing embeddings in a vector database reduces search time and improves response speed.

Caching frequently accessed responses can further reduce processing time and improve user experience.

Efficient workflow execution through LangGraph also helps minimize unnecessary processing steps.

5.4 Storage Optimization

As the number of stored embeddings increases, efficient storage management becomes important.

The system uses vector compression techniques and efficient storage formats to reduce memory usage. Periodic cleanup of unused data can also help maintain storage efficiency.

Cloud-based storage solutions may be used in future implementations to handle large-scale deployments.

5.5 Future Scalability Enhancements

The system can be extended to support additional features such as:

Multi-document support

Multi-language support

Web-based user interface

Integration with enterprise systems

Distributed processing using cloud infrastructure

These enhancements will allow the system to support larger datasets and more users.